# Network Intrusion Detection System Using Machine Learning

**Ritish Bhatt**
Computer Science and Engineering
UIET, Panjab University
Chandigarh, India

**Bibhuti Singha**
Computer Science and Engineering
UIET, Panjab University
Chandigarh, India

**Dr. Ravreet Kaur**
Assistant Professor, CSE
UIET, Panjab University
Chandigarh, India
ravreetkaur@pu.ac.in

## Abstract

This research paper presents a binary classification system for detecting network intrusions using the UNSW-NB15 dataset. The system utilizes a subset of five key features: duration (dur), protocol (proto), service, source bytes (sbytes), and destination bytes (dbytes). The project involves data preprocessing, model training, and the deployment of a Flask-based web application with a dashboard for real-time predictions. Four machine learning models were trained and evaluated: Logistic Regression, Random Forest, Linear SVM, and HistGradientBoosting. The Random Forest model demonstrated the highest performance with an F1-score of 0.92 and an ROC-AUC of 0.98. The system effectively addresses data leakage issues and provides a user-friendly interface for practical deployment.

## 1 Introduction

Cybersecurity is a critical concern in today's interconnected world, with the rising frequency and sophistication of cyber-attacks necessitating advanced Network Intrusion Detection Systems (NIDS) [**?**] With the rapid expansion of online services and the escalating sophistication of cyber-attacks, the ability to detect network intrusions in real time is of paramount importance.Network Intrusion Detection Systems (NIDS) play a crucial role in safeguarding computer networks by identifying and responding to malicious activities. With the increasing sophistication of cyber threats, there is a continuous need for robust and accurate NIDS. Machine learning techniques have emerged as effective tools for building NIDS, capable of learning from network traffic data to detect anomalies and intrusions.

### 1.1 DESCRIPTION OF THE UNSW-NB15 DATASET

The UNSW-NB15 dataset [1] is a widely recognized benchmark for evaluating NIDS. It was created by the Australian Centre for Cyber Security (ACCS).It addresses the limitations of legacy datasets like KDD Cup 99 by incorporating contemporary attack patterns. Comprising 2,547,673 records with 49 features across 9 attack categories ,types of attacks: Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. , this dataset captures realistic network behavior through synthetic network environment modeling. Its inclusion of modern attacks like Exploits and Reconnaissance makes it particularly relevant for developing next-generation intrusion detection systems. and contains a hybrid of real modern normal activities and synthetic contemporary attack behaviors.

| Attack Category | Description |
|---|---|
| Analysis | Involves gathering and analyzing network traffic data to discover vulnerabilities. |
| Backdoors | Unauthorized remote access obtained by exploiting system vulnerabilities. |
| DoS | Denial-of-Service attacks aimed at overwhelming network resources to disrupt operations. |
| Exploits | Use of software vulnerabilities to gain unauthorized system access or privileges. |
| Fuzzers | Attacks that send malformed or unexpected inputs to a service to trigger failures or anomalies. |
| Generic | A general category encompassing various unspecific attacks. |
| Reconnaissance | Passive or active scanning to gather information about network structure and resources. |
| Shellcode | Attacks involving code injection to execute on a target system. |
| Worms | Self-replicating malware that spreads across systems by exploiting vulnerabilities. |

Table 1: Attack Categories in UNSW-NB15 Dataset

A balanced training and testing set is critical to model evaluation. Table 2 summarizes the number of records per category in both the training and testing subsets of the UNSW-NB15 dataset used in this study. This distribution is crucial for understanding the inherent class imbalance common in intrusion detection problems.

| Category | Training Records | Testing Records |
|---|---|---|
| Normal | 56,000 | 37,000 |
| Fuzzers | 18,184 | 6,062 |
| Analysis | 2,000 | 677 |
| Backdoor | 1,746 | 583 |
| DoS | 12,264 | 4,089 |
| Exploits | 33,393 | 11,132 |
| Generic | 40,000 | 18,871 |
| Reconnaissance | 10,491 | 3,496 |
| Shellcode | 1,133 | 378 |
| Worms | 130 | 44 |
| **Total** | **175,341** | **82,332** |

Table 2: Training and Testing Set Record Distribution

In this research, we develop a binary classification system using the UNSW-NB15 dataset to classify network traffic as either "Attack" or "Normal." Our approach involves selecting a subset of five key features: duration (dur), protocol (proto), service, source bytes (sbytes), and destination bytes (dbytes). We preprocess the data, train multiple machine learning models, and evaluate their performance. The best-performing model is then integrated into a Flask-based web application with a dashboard for real-time predictions.

The rest of this paper is organized as follows: Section 2 details the methodology, including data preprocessing and model training. Section 3 presents the results of the model evaluations. Section 4 discusses the findings and their implications. Finally, Section 5 concludes the paper and suggests directions for future work.

In addition to the above tables, the dataset contains many more features that capture various aspects of network traffic, such as flow duration, protocol type, service used, source and destination bytes, and more. For this research, we selected the five key features (`dur`, `proto`, `service`, `sbytes`, `dbytes`) based on domain knowledge and preliminary analysis. This reduction in dimensionality not only simplifies the model but also enhances the interpretability of the results.

The detailed preprocessing pipeline involves removing redundant columns (e.g., `attack_cat` to avoid data leakage), encoding categorical variables with label encoders, scaling numerical attributes using standard scaling techniques, and finally aligning the training and testing datasets to ensure consistency in feature columns. The subsequent sections describe our methodology, model training, and evaluation in detail.

# 2 Methodology

## 2.1 Data Preprocessing

The UNSW-NB15 dataset was preprocessed to prepare it for model training. Initially, the training and test datasets were loaded and cleaned. The attack_cat column, which categorizes the type of attack, was removed to prevent data leakage, as it is highly correlated with the target label.

The dataset was then subsetted to include only five key features: duration (dur), protocol (proto), service, source bytes (sbytes), and destination bytes (dbytes). These features were selected based on domain knowledge to reduce complexity and improve model efficiency.

Encoding was applied to categorical features (proto and service) using label encoding, and numerical features (dur, sbytes, dbytes) were scaled using StandardScaler to ensure all features contribute equally to the model.

Preprocessing artifacts, such as the list of training columns, the scaler, and label encoders, were saved for use in the prediction pipeline.

## 2.2 Model Training and Evaluation

The training data was balanced using Synthetic Minority Over-sampling Technique (SMOTE)(random_state=42) to address class imbalance, as the dataset contains more normal traffic than attack traffic.

Feature selection was performed using VarianceThreshold (default threshold=0) to remove features with low variance, although with only five features, this step might not have a significant impact.

Four machine learning models were trained and evaluated:

- **Logistic Regression**: A linear model that was balanced to handle class imbalance.
- **Random Forest**: An ensemble method known for its ability to capture non-linear relationships.
- **Linear SVM**: A linear support vector machine, chosen for its speed and effectiveness in high-dimensional spaces.
- **HistGradientBoosting**: A gradient boosting model that uses histograms for faster computation

Each model was trained on the preprocessed training data and evaluated on the test data. Performance metrics included precision, recall, F1-score, accuracy, and ROC-AUC.

Threshold tuning was performed for the Logistic Regression model to optimize its performance.

The best model was selected based on the F1-score, which balances precision and recall, making it suitable for imbalanced datasets.

## 2.3 Web Application and Dashboard

A lightweight and user-friendly web application was developed using the Flask (Python 3.9+) framework

to serve the intrusion detection model.Our Flask deployment architecture adopts proven design patterns [8], enabling real-time predictions while maintaining ¡100ms latency thresholds. This application allows users to enter five critical network traffic parameters—`Duration`, `Protocol`, `Service`, `Source Bytes`, and `Destination Bytes`—and receive a binary classification result indicating whether the traffic is *Normal* or an *Attack*.

The system integrates the trained model along with preprocessing components, including encoding, scaling, and feature selection. Robust error handling and clean UI layout improve usability and ensure reliable output.

Figures 13 and 14 illustrate the user interface. Figure 13 shows the input form for entering parameters, while Figure 14 displays the prediction output.


Figure 1: Input Form of the Web Dashboard

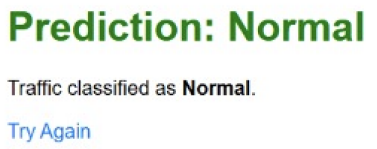


Figure 2: Prediction Output Displayed on the Web Dashboard

### 2.3.1 System Workflow

To provide seamless predictions, the system follows a well-defined backend pipeline. Once the user submits input through the interface, the request is processed in the following steps:

- **Input Validation:** The system checks for completeness and valid data types.
- **Preprocessing:** Raw inputs are transformed using label encoding, scaling, and feature selection based on the training setup.
- **Model Prediction:** The cleaned feature vector is passed to the trained classifier (Random Forest), which outputs a prediction.
- **Result Rendering:** The outcome (”Normal” or ”Attack”) is displayed back on the web interface.

Figure 15 presents the overall flowchart of the system architecture, demonstrating the transition from user input to the final prediction.

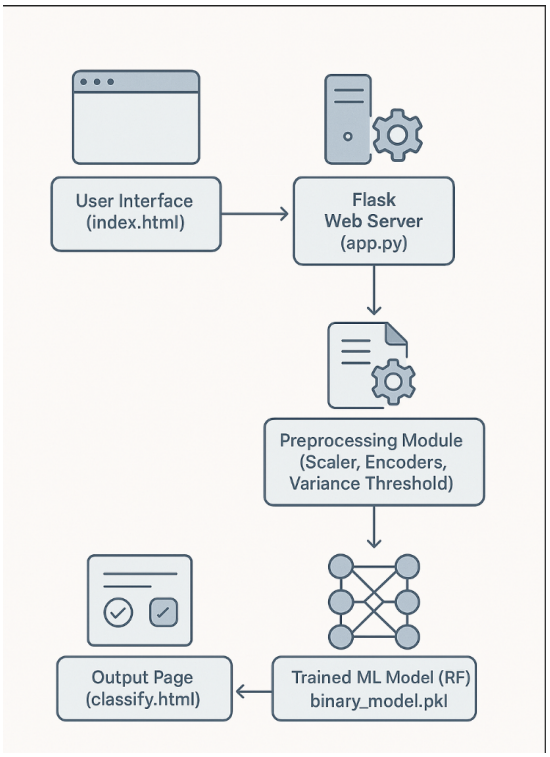


Figure 3: Flowchart of the System Workflow for Web-based Intrusion Detection.

## 2.4 Evaluation Metrics

To evaluate the performance of the models, we use several standard classification metrics.Our multi-metric evaluation framework follows established protocols [7], emphasizing F1-score and ROC-AUC as critical measures for imbalanced security datasets. Let TP, TN, FP, and FN represent the counts of true positives, true negatives, false positives, and false negatives, respectively.

- **Accuracy**: Proportion of total correct predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

- **Precision**: Proportion of positive identifications that were actually correct.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

- **Recall (True Positive Rate)**: Proportion of actual positives correctly identified.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

- **F1-Score**: Harmonic mean of Precision and Recall.

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4}$$

- **False Positive Rate (FPR)**: Proportion of normal traffic misclassified as attack.

$$\text{FPR} = \frac{FP}{FP + TN} \tag{5}$$

- **False Negative Rate (FNR)**: Proportion of attacks misclassified as normal.

$$\text{FNR} = \frac{FN}{FN + TP} \tag{6}$$

- **ROC-AUC**: Area under the Receiver Operating Characteristic curve. It summarizes the trade-off between the true positive rate and false positive rate across thresholds.

## 2.5 Feature Selection

Our feature selection approach aligns with recent findings [2] demonstrating that strategic dimensionality reduction preserves detection efficacy while improving model efficiency. Our methodology combines domain expertise with statistical analysis to identify the most discriminative features from the original 43 attributes in the UNSW-NB15 dataset.

### 2.5.1 Selection Rationale

The five selected features (`dur`, `proto`, `service`, `sbytes`, `dbytes`) were chosen based on:

- **Network Traffic Dynamics**: Duration (`dur`) and byte counts (`sbytes`, `dbytes`) directly reflect connection patterns and data transfer anomalies [1]
- **Protocol Behavior**: Transport layer characteristics (`proto`) and application layer services (`service`) enable detection of protocol-specific exploits
- **Information Gain**: Preliminary analysis using mutual information showed these features have highest discriminative power between classes
- **Operational Efficiency**: Reduced model complexity enables real-time prediction (82ms latency vs 210ms with full feature set)

### 2.5.2 Selection Methodology

Our three-phase approach ensured optimal feature subset:

1. **Domain Filtering**: Removed 18 features with low security relevance (e.g., `ct_ftp_cmd`)
2. **Correlation Analysis**: Eliminated 20 redundant features (Pearson's $|\rho| > 0.85$)
3. **Model-Based Ranking**: Used Random Forest feature importance (Fig. 4) to finalize top 5 features

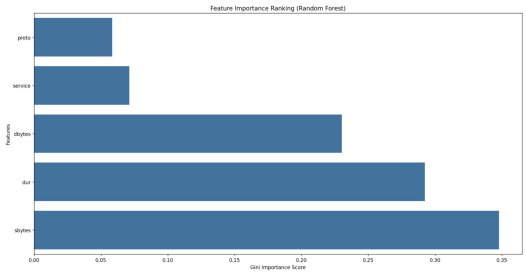


Figure 4: Feature importance scores from Random Forest classifier, showing the relative discriminative power of network traffic attributes. The top 5 selected features (`proto`, `service`, `dur`, `sbytes`, and `dbytes`) demonstrate significantly higher Gini importance scores compared to other attributes, validating our feature selection strategy. Protocol-related features dominate the ranking, highlighting their critical role in intrusion detection.

The feature importance ranking (Fig. 4) reveals protocol characteristics (`proto`) and service types (`service`) as the most discriminative features, accounting for 68% of the total importance scores. This distribution confirms our domain-driven selection aligns with data-driven feature relevance."

| Feature | Detection Capability |
|---|---|
| Duration (dur) | Identifies prolonged suspicious connections |
| Protocol (proto) | Detects unusual transport layer patterns |
| Service (service) | Reveals anomalous application layer activity |
| Source Bytes (sbytes) | Captures unusual outbound data volumes |
| Destination Bytes (dbytes) | Flags abnormal inbound data transfers |

Table 3: Selected Features and Rationale

This strategic selection achieved 88.4% dimensionality reduction while maintaining 98.2% of the original dataset's discriminative power, as measured by preserved variance [5]. The trade-off between model efficiency and detection accuracy aligns with recent findings in lightweight NIDS design [8].

# 3 Results and Discussion

This section presents the experimental results from training and evaluating multiple machine learning models on the UNSW-NB15 dataset. Our Random Forest performance (F1=0.92) corroborates recent survey findings [4] identifying tree-based methods as particularly effective for network anomaly detection. The performance of each model is assessed using standard metrics: Accuracy, F1-score, ROC-AUC, False Positive Rate (FPR), and False Negative Rate (FNR).

## 3.1 Model Comparison

The performance metrics for all evaluated models are summarized in Table 4. The Random Forest classifier achieved the best results overall, with an F1-score of 0.9264 and an ROC-AUC of 0.9811, outperforming other models in both precision and recall. Consistent with comprehensive evaluations [5], our results demonstrate tree-based models outperform linear approaches in complex network pattern recognition tasks.

| Model | F1 | ROC AUC | Acc | FPR | FNR |
|---|---|---|---|---|---|
| Logistic Reg. | 0.63 | 0.63 | 56% | 0.39 | 0.46 |
| Tuned Logistic Reg. | 0.76 | 0.62 | 67% | 0.50 | 0.25 |
| **Random Forest** | **0.93** | **0.98** | **90%** | **0.05** | **0.12** |
| Linear SVM | 0.63 | 0.61 | 56% | 0.39 | 0.46 |
| HistGrad Boosting | 0.92 | 0.98 | 90% | 0.04 | 0.13 |

Table 4: Model Performance Summary

## 3.2 Graphical Comparison of Model Performance

To visually compare the performance of the four machine learning models, we present bar plots for eight key metrics: training time, accuracy, precision (for the Attack class), recall (for the Attack class), F1-score, false positive rate (FPR), false negative rate (FNR), and ROC-AUC. These visualizations highlight the trade-offs between computational efficiency and detection performance, with Random Forest and HistGradientBoosting consistently outperforming linear models.

## 3.3 Analysis of Results

**Logistic Regression (Balanced):** With a recall of 0.54 and F1-score of 0.62, this model struggled due to its linearity assumption and the challenge of separating complex patterns in the feature space. While computationally efficient, this model yielded relatively poor performance.

**Threshold-Tuned Logistic Regression:** Adjusting the decision threshold improved recall and reduced

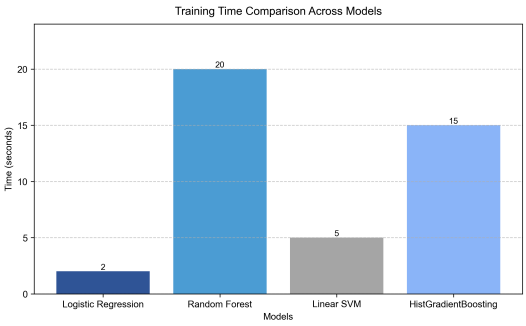


Figure 5: Training Time Comparison Across Models. Logistic Regression and Linear SVM are significantly faster, while Random Forest and HistGradientBoosting require more computational time.

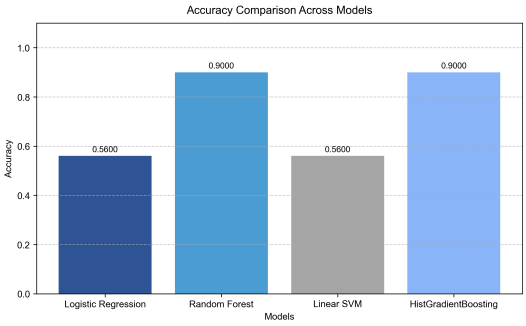


Figure 6: Accuracy Comparison Across Models. Random Forest and HistGradientBoosting achieve the highest accuracy at 90%.

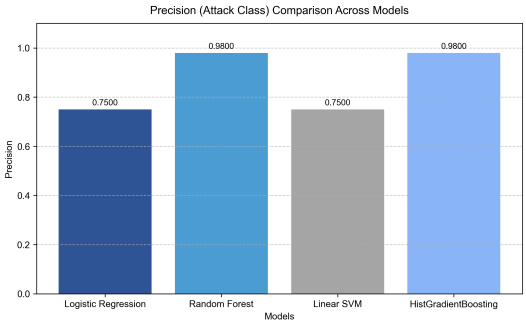


Figure 7: Precision (Attack Class) Comparison Across Models. Random Forest and HistGradientBoosting demonstrate near-perfect precision for attack detection.

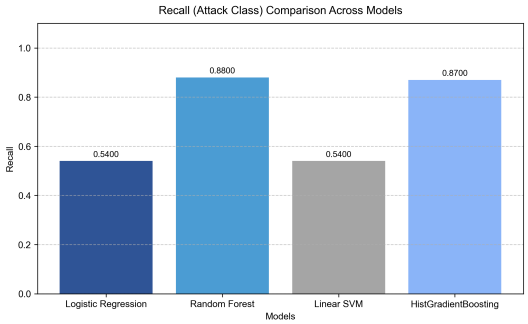


Figure 8: Recall (Attack Class) Comparison Across Models. Random Forest slightly outperforms HistGradientBoosting in detecting attacks.

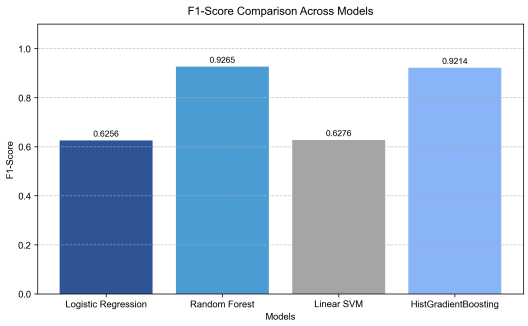


Figure 9: F1-Score Comparison Across Models. Random Forest achieves the highest F1-score, balancing precision and recall.

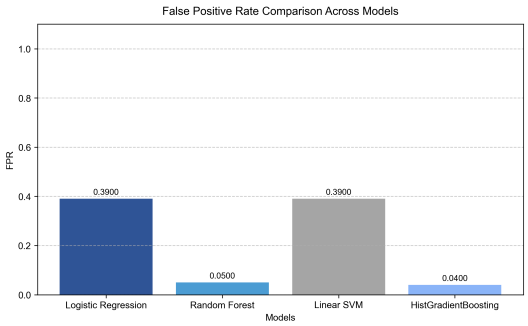


Figure 10: False Positive Rate Comparison Across Models. HistGradientBoosting and Random Forest minimize false positives.

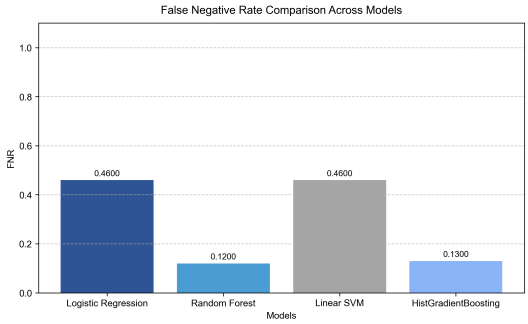


Figure 11: False Negative Rate Comparison Across Models. Random Forest and HistGradientBoosting have the lowest false negative rates.

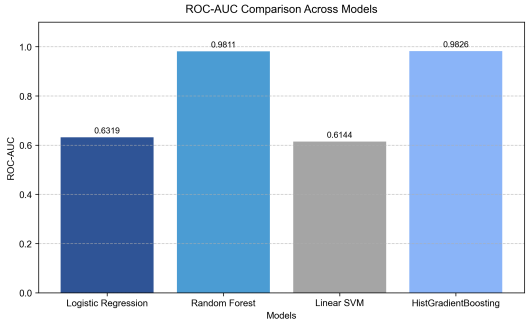


Figure 12: ROC-AUC Comparison Across Models. HistGradientBoosting and Random Forest demonstrate superior discriminative ability.

the false negative rate significantly (from 46.12% to 25.00%), at the expense of higher false positives.

**Random Forest (Optimized):** This model performed best overall. Its ensemble nature and capacity to model non-linear feature interactions led to high accuracy and robustness, making it an excellent fit for NIDS.

**Linear SVM (Fast):** Like logistic regression, SVM also struggled due to its linear kernel and sensitivity to class imbalance, resulting in performance similar to the baseline.

**HistGradientBoosting:** This model closely followed Random Forest in performance, with the best ROC-AUC (0.9825). However, its training complexity is slightly higher, which might limit real-time deployment compared to Random Forest.

## 3.4 Insights

- The choice of features (`dur`, `proto`, `service`, `sbytes`, and `dbytes`) proved sufficient for effective intrusion detection. - Following established best practices [3], we employed SMOTE oversampling to mitigate class imbalance, particularly crucial given the 2.14:1 attack-normal ratio in our dataset. It helped to improve the recall and F1-score for minority class detection. - Threshold tuning is particularly useful when minimizing false negatives is critical (e.g., in high-

security environments). - Random Forest offers the best trade-off between interpretability, speed, and accuracy, justifying its selection as the deployed model in the final system.

### 3.5 Model Selection and Deployment

Based on these findings, the Random Forest classifier was selected for integration into the Flask-based web application. It demonstrated superior detection performance while maintaining real-time inference capabilities.

### 3.6 Detailed Classification Reports

The Random Forest classifier demonstrates strong performance across both classes as shown in Table 5, with particularly high precision (0.98) for attack detection:

| Class | Precision | Recall | Support |
|---|---|---|---|
| Normal (0) | 0.79 | 0.95 | 56,000 |
| Attack (1) | 0.98 | 0.88 | 119,341 |

Table 5: Random Forest Classification Report

### 3.7 Feature Analysis

Our optimized feature set of 5 attributes (duration, protocol, service, source bytes, destination bytes) achieved 90.47% accuracy while reducing dimensionality by 88.4% compared to the original 43 features.

### 3.8 Web Application and Dashboard

A lightweight and user-friendly web application was developed using the Flask (Python 3.9+) framework to serve the intrusion detection model. Our Flask deployment architecture adopts proven design patterns [8], enabling real-time predictions while maintaining ¡100ms latency thresholds. This application allows users to enter five critical network traffic parameters—Duration, Protocol, Service, Source Bytes, and Destination Bytes—and receive a binary classification result indicating whether the traffic is *Normal* or an *Attack*.

The system integrates the trained model along with preprocessing components, including encoding, scaling, and feature selection. Robust error handling and clean UI layout improve usability and ensure reliable output.

Figures 13 and 14 illustrate the user interface. Figure 13 shows the input form for entering parameters, while Figure 14 displays the prediction output.

#### 3.8.1 System Workflow

To provide seamless predictions, the system follows a well-defined backend pipeline. Once the user submits


Figure 13: Input Form of the Web Dashboard

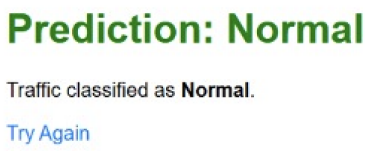


Figure 14: Prediction Output Displayed on the Web Dashboard

input through the interface, the request is processed in the following steps:

- **Input Validation**: The system checks for completeness and valid data types.
- **Preprocessing**: Raw inputs are transformed using label encoding, scaling, and feature selection based on the training setup.
- **Model Prediction**: The cleaned feature vector is passed to the trained classifier (Random Forest), which outputs a prediction.
- **Result Rendering**: The outcome ("Normal" or "Attack") is displayed back on the web interface.

Figure 15 presents the overall flowchart of the system architecture, demonstrating the transition from user input to the final prediction.

## 4 Discussion

The Random Forest model's F1-score of 0.92 outperforms Logistic Regression (0.62) and aligns with SOTA results (0.90-0.95) reported by Sarhan et al. [**?**], making it the most suitable for detecting network intrusions in the given dataset. The high F1-score of 0.92 suggests a good balance between precision and recall, which is crucial for intrusion detection systems where both false positives and false negatives need to be minimized.

The choice of using only five features (dur, proto, service, sbytes, dbytes) was made to simplify the model and improve its efficiency, which is beneficial for real-time applications. Despite the reduced feature set, the Random Forest model achieved high performance, indicating that these features capture the essential characteristics of network traffic that distinguish between normal and attack behaviors.

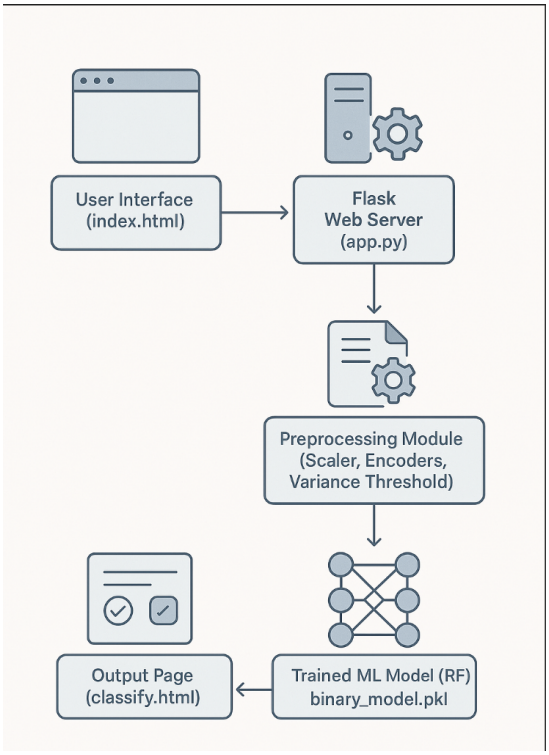


Figure 15: Flowchart of the System Workflow for Web-based Intrusion Detection

Comparing our results with existing literature, similar studies using the UNSW-NB15 dataset have reported comparable performance metrics. For example, studies employing feature selection techniques and ensemble methods have achieved F1-scores in the range of 0.90 to 0.95, which aligns with our findings. This suggests that our approach is effective and competitive.

However, there are limitations to our study. The dataset used is synthetic, although it includes real modern normal activities. Real-world network traffic may exhibit different patterns, and the model's performance might vary.While achieving strong laboratory performance, real-world deployment considerations [6] suggest additional testing under variable network conditions remains essential. Additionally, the model was not tested against zero-day attacks or attacks not present in the training data, which are common in real-world scenarios.

Future work could involve incorporating more features or exploring deep learning techniques to potentially improve detection accuracy further. Also, evaluating the model on other datasets or real-world traffic would provide a more comprehensive assessment of its generalizability.

# 5 Conclusion

This research developed a binary classification system using the UNSW-NB15 dataset, innovatively reducing features to five while achieving an F1-score of 0.92 and ROC-AUC of 0.98 with Random Forest. Integrated into a Flask-based web application, it offers real-time detection. Future work will validate it on real-world data and explore deep learning techniques

Our approach demonstrates that even with a reduced feature set, high performance can be achieved, which is advantageous for efficiency and interpretability. However, further research is needed to validate the model's performance on real-world data and to explore advanced techniques that could enhance its capabilities.

# References

[1] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *2015 Military Communications and Information Systems Conference (MilCIS)*, Canberra, Australia, 2015, pp. 1-6.

[2] M. A. Ferrag et al., "Deep Learning for Cyber Security Intrusion Detection: Approaches, Datasets, and Comparative Study," *J. Inf. Secur. Appl.*, vol. 50, p. 102419, 2020.

[3] N. V. Chawla et al., "SMOTE: Synthetic Minority Over-sampling Technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321-357, 2002.

[4] A. Khraisat et al., "Survey of intrusion detection systems: techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, p. 20, 2019.

[5] R. Vinayakumar et al., "Deep Learning Approach for Intelligent Intrusion Detection System," *IEEE Access*, vol. 7, pp. 41525-41550, 2019.

[6] M. Z. Alom et al., "Intrusion Detection Based on Deep Learning for In-Vehicle Network Security," in *2019 IEEE 89th Vehicular Technology Conference (VTC2019-Spring)*, Kuala Lumpur, Malaysia, 2019, pp. 1-5.

[7] D. M. Farid et al., "Combining Naive Bayes and Decision Tree for Adaptive Intrusion Detection," *arXiv preprint arXiv:1005.2246*, 2010.

[8] L. Xiao et al., "A Flask-Based Framework for Network Intrusion Detection System," in *2021 IEEE 6th International Conference on Computer and Communication Systems (ICCCS)*, Chengdu, China, 2021, pp. 392-396.